P3288R0
std::elide

New Proposal, 2024-05-17

This version:
http://virjacode.com/papers/p3288r0.htm

Latest version:
http://virjacode.com/papers/p3288.htm

Author:
TPK Healy <healytpk@vir7ja7code7.com> (*Remove all sevens from email address*)

Audience:
SG17, SG18

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Add a new class to the standard library to make possible the emplacement of an unmovable-and-uncopyable *prvalue* into types such as `std::optional`, `std::variant`, `std::any` -- without requiring an alteration to the definitions of these classes.

Table of Contents

NOTE: A change is required to the core language.

§ 1. Introduction

While it is currently possible to return an unmovable-and-uncopyable class by value from a function:

```
std::counting_semaphore<8> FuncReturnsByValue(unsigned const a, unsigned co
{
    return std::counting_semaphore<8>(a + b);
}
```

It is not possible to emplace this return value into an `std::optional`:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    var.emplace( FuncReturnsByValue(1,2) );  // compiler error
}
```

This paper proposes a solution to this debacle, involving an addition to the standard library, along with a change to the core language.

§ 2. Motivation

§ 2.1. `emplace( FuncReturnsByValue() )`

There is a workaround to make this possible, and it is to use a helper class with a conversion operator:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    struct Helper {
        operator std::counting_semaphore<8>()
        {
            return FuncReturnsByValue(1,2);
        }
    };

    var.emplace( Helper() );
}
```

This is possible because of how the `emplace` member function is written:

```
template<typename... Params>
T &emplace(Params&&... args)
{
    . . .
    ::new(buffer) T( forward<Params>(args)... );
    . . .
}
```

The compiler cannot find a constructor for `T` which accepts a sole argument of type `Helper`, and so it invokes the conversion operator, meaning we effectively have:

```
::new(buffer) T( FuncReturnsByValue(1,2) );
```

In this situation, where we have a *prvalue* returned from a function, we have guaranteed elision of a copy/move operation. This proposal aims to simplify this technique by adding a new class to the standard library called `std::elide` which can be used as follows:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    var.emplace( std::elide(FuncReturnsByValue,1,2) );
}
```

§ 3. Possible implementation

```
#include <functional>   // invoke
#include <tuple>        // apply, tuple
#include <type_traits>  // invoke_result, is_same, remove_reference
#include <utility>      // move

namespace std {
  template<typename F_ref, typename... Params_refs>
  class elide final {
      using R = invoke_result_t< F_ref, Params_refs... >;
      static_assert( is_same_v< R, remove_reference_t<R> > );  // F must re
      using F = remove_reference_t<F_ref>;
      F &&f;  // 'f' is always an Rvalue reference
      tuple< Params_refs... > const args_tuple;  // just a tuple full of re
  public:
      template<typename F, typename... Params>
      explicit elide(F &&arg, Params&&... args) noexcept  // see explicit c
        : f(move(arg)), args_tuple( static_cast<Params&&>(args)... ) {}

      operator R(void) noexcept(noexcept(apply(static_cast<F_ref>(f),move(a
      {
          return apply( static_cast<F_ref>(f), move(args_tuple) );
      }

      /* -------- Delete all miranda methods -------- */
      elide(     void     ) = delete;
      elide(elide const & ) = delete;
      elide(elide       &&) = delete;
```

```
        elide &operator=(elide const & ) = delete;
        elide &operator=(elide       &&) = delete;
        elide const volatile *operator&(void) const volatile = delete;
        template<typename U> void operator,(U&&) = delete;
        /* ------------------------------------------ */
    };

    template<typename F, typename... Params>
    elide(F&&,Params&&...) -> elide<F&&,Params&&...>;  // explicit deduction
}
```

Thoroughly tested on GodBolt: https://godbolt.org/z/65fEd3axf
You can comment out **Line #85** in the godbolt to see the effect of the core language change.

§ 4. Design considerations

§ 4.1. template constructor

The above implementation of `std::elide` will not work in a situation where a class has a constructor which accepts a specialisation of the template class `std::elide` as its sole argument, such as the following `AwkwardClass`:

```
class AwkwardClass {
    std::mutex m;  // cannot move, cannot copy
public:
    template<typename T>
    AwkwardClass(T &&arg)
    {
        cout << "In constructor for AwkwardClass, \n"
                "type of T = " << typeid(T).name() << endl;
    }
};

AwkwardClass ReturnAwkwardClass(int const arg)
{
    return AwkwardClass(arg);
}

int main(int const argc, char **const argv)
```

```
{
    std::optional<AwkwardClass> var;
    var.emplace( std::elide(ReturnAwkwardClass, -1) );
}
```

This program will print out:

```
--
In constructor for AwkwardClass,
type of T = std::elide< AwkwardClass, AwkwardClass (&)(int), int&& >
--
```

The problem here is that the constructor of `AwkwardClass` has been instantiated with the template parameter type `T` set to a specialisation of `std::elide`, when really we wanted `T` to be set to `int`. We want the following output:

```
--
In constructor for AwkwardClass,
type of T = int
--
```

A workaround here is to apply a constraint to the constructor of `AwkwardClass` as follows:

```
template<typename T>
requires (!is_specialization_of_v< std::remove_cvref_t<T>, std::elide >)
AwkwardClass(T &&arg)
{
    cout << "In constructor for AwkwardClass, \n"
            "type of T = " << typeid(T).name() << endl;
}
```

In order that class definitions do not have to be altered in order to apply this constraint to template constructors, this proposal makes a change to the core language to prevent the constructor of any class type from having a specialisation of `std::elide` as any of its parameter types.

§ 5. Proposed wording

The proposed wording is relative to [N4950].

In subclause 13.10.3.1.11 [temp.deduct.general], append a paragraph under the heading *"Type deduction can fail for the following reasons:"*

```
11 -- Attempting to instantiate a constructor in which a parameter
      has a type which is a specialization of std::elide.
```

§ 6. Impact on the standard

This proposal is a library extension combined with a change to the core language. The change to the core language is a paragraph to be added to 13.10.3.1.11 [temp.deduct.general]. The addition has no effect on any other part of the standard.

§ 7. Impact on existing code

No existing code becomes ill-formed. The behaviour of all existing code is unaffected.

§ 8. Acknowledgements

For their feedback and contributions on the mailing list **std-proposals@lists.isocpp.org**:

Jens Mauer, Ville Voutilainen, Jonathan Wakely

And for their insightful blogs:

Andrzej Krzemieński, Arthur O'Dwyer

§ References

§ Normative References

[N4950]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: https://wg21.link/n4950